Profile-Driven Inference Optimization of a Compact Vision-Language Model for Brain MRI Classification

Team Members

Team 32: Owen Shi (as8254), Chengshun Tan (ct3013), Ai Hsiao (ah4617), Trung Kien Giang (tg3049).

- Owen Shi: data pipeline, deduplication, splits; preprocessing/transfer optimizations; quality validation for all configurations.
- Chengshun Tan: benchmarking harness, FP16 baseline, stage-level profiling and bottleneck analysis; vision-encoder/prefill optimizations.
- Ai Hsiao: LoRA fine-tuning and checkpoint merge; weight quantization and memory analysis.
- Trung Kien Giang: W&B tracking, repository, environment and Slurm reproducibility; decode-stage optimizations; report assembly.

Note: Current task assignments, subjected to change throughout the development of the project.

Collaboration: weekly team meeting plus a group chat for day-to-day issues; every member reviews others' pull requests and presents part of the final talk. Disagreements are settled by discussion, then a team vote, with instructor office hours for ties or scope questions.

Goal / Objective

We will study where inference time and memory go when a compact vision-language model (VLM) classifies brain tumor MRI, and let that profile choose the optimizations we apply.

Claim: On one fixed GPU at the fixed batch size, INT8/INT4 weight quantization will reduce peak inference memory but not per-image latency relative to the merged FP16 LoRA baseline. A configuration chosen from the profile to target the dominant stage will reduce per-image latency, while all configurations stay within 2 macro-F1 points of the baseline on the test set.

Questions:

1. Which stage (preprocessing, data transfer, prefill, vision encoder, decoder) limits latency, and is it compute-bound or memory-bound?
2. Does INT8/INT4 quantization or a smaller image-token budget address that bottleneck without sacrificing accuracy?

Challenges

- **Domain gap and small data.** General VLMs are weak on MRI zero-shot and may overfit when fine-tuned, so we fine-tune once with LoRA to get a meaningful quality baseline and track validation loss.
- **Optimization overhead and compatibility.** Quantized inference may save memory without reducing latency, and support depends on the available GPU and software [6]. We will check compatibility early, compare supported configurations with the merged FP16 baseline, and use profiling to explain any speedup or slowdown.
- **Measurement noise and reproducibility.** Different hardware or environments across teammates would corrupt timings, so all before/after runs use one fixed GPU type and a pinned environment, with warm-up and repeated trials.

Approach and Performance Optimization Techniques

Experiment steps:

1.Baseline → 2. Profile → 3.Optimize → 4.Re-measure → 5.Validate → 6.Explain

Environment: on a deliberately small scope with one primary VLM (SmolVLM-500M-Instruct [2]); SmolVLM2-2.2B [3] only if the baseline is running by the midpoint; a fine-tuned ResNet-18 [4] as a single accuracy reference row.

- **Baseline:** zero-shot evaluation, then one LoRA fine-tune [5]; the adapter is merged into FP16 weights, and we sweep batch sizes on this model to pick one, fixed before optimizing, for all later comparisons.
- **Profile:** we will use PyTorch Profiler [8] to examine the time spent in image preprocessing, CPU-GPU data transfer, the vision encoder, prefill and decoding, alongside memory usage and GPU utilization. The results will guide our choice of optimizations. We will then repeat the profiling to explain whether and why performance changed.
- **Optimize (at most 2–3, chosen from the profile):** We will choose techniques that target the dominant measured costs: parallel CPU preprocessing for image preparation; pinned-memory and non-blocking copies for CPU-GPU transfer; lower supported resolution or fewer image crops for vision encoding; fewer image tokens or a shorter prompt for prefill; and label-only generation or class-label scoring for decoding. If model weights dominate GPU memory, we will also consider 8-bit or 4-bit quantization with bitsandbytes [6]. We will change one factor at a time using the same merged LoRA checkpoint, keep other settings fixed, and select configurations on validation data. We will re-profile and measure end-to-end latency and memory to confirm benefits while meeting the agreed quality threshold.
- **Re-measure and validate:** we will compare each optimized configuration with the merged FP16 baseline on the same GPU and input images at the fixed baseline batch size. After 10 warm-up runs, we will take 50 timed measurements and report mean, p50 and p95 latency, throughput and peak GPU memory, with repeated trials and recorded seeds. Configurations and the macro-F1 tolerance will be fixed using validation data before final testing. We will report test macro-F1 and per-class recall alongside performance changes; only improvements meeting the quality threshold will count toward our claim.
- **Explain:** for each result, state which resource was limiting and why the change did or did not help.

Implementation Details

- **Hardware:** Columbia's Insomnia cluster with Slurm scheduling [7]; one GPU type held fixed for every before/after run, with the exact GPU recorded. No paid compute services are necessary.
- **Software:** We will use PyTorch and Hugging Face Transformers for SmolVLM loading and inference, PEFT for LoRA, bitsandbytes for quantization, and torchvision for the ResNet-18 reference. PyTorch Profiler [8] and, if needed, Nsight Systems will support profiling.
- **Dataset:** Nickparvar Brain Tumor MRI Dataset (glioma, meningioma, pituitary, no tumor) [1], using the provided test set and splitting the train set into train + validation. Before training, we will remove near-duplicate images (perceptual hash) that appear across train and test, so the matched-accuracy check is not inflated by data leakage.
- **Experiment tracking:** We will use a shared Weights & Biases project [9] to record benchmark configurations and results. Each benchmark run will log hardware, precision, batch size, timing settings, the corresponding baseline, and quality/performance metrics. Selected profiler traces will be uploaded as artifacts. The repository will include environment setup, reproduction instructions, and Slurm scripts where applicable.

Demo Planned

We will demonstrate baseline and optimized inference on the same held-out MRI examples, comparing predictions and measured latency and memory. We will interpret profiler traces to explain the observed bottleneck and the effect of optimization, and use our W&B dashboard to present aggregate performance and classification-quality results. A Gradio interface may be added if time permits.

References

[1] Nickparvar, Brain Tumor MRI Dataset.
https://www.kaggle.com/datasets/masoudnickparvar/brain-tumor-mri-dataset

[2] Hugging Face, SmolVLM-500M-Instruct. https://huggingface.co/HuggingFaceTB/SmolVLM-500M-Instruct

[3] Hugging Face, SmolVLM2-2.2B-Instruct. https://huggingface.co/HuggingFaceTB/SmolVLM2-2.2B-Instruct

[4] PyTorch, ResNet-18.
https://docs.pytorch.org/vision/stable/models/generated/torchvision.models.resnet18.html

[5] Hugging Face PEFT, LoRA documentation. https://huggingface.co/docs/peft/main/en/conceptual_guides/lora

[6] Hugging Face Transformers, Bitsandbytes quantization.
https://huggingface.co/docs/transformers/main/quantization/bitsandbytes

[7] Columbia University, Research Computing. https://www.cuit.columbia.edu/research-computing

[8] PyTorch Profiler documentation. https://docs.pytorch.org/docs/stable/profiler.html

[9] Weights & Biases. https://wandb.ai